

CS141 Coursework Report: LGPT

Szymon Galutowski / 5605010

1 Introduction

My particular instance of LGPT works completely as expected, albeit with some light quality of life changes, and one, perhaps two major extensions. One of them is the usage of the `Network.HTTP.Simple`, `Data.Aeson`, and `GHC.Generics` packages in order to access the **Dictionary API**, and allow the bot to define words. The other major extension is the ability to output the contents of system files, and also upload its own memory into a specific one.

To find the packages and documentation I needed, I used Hackage.

Since I deem this to be the most important, I will now go into how the memory works.

2 Memory

The `TUI.runREPL` function is of the type `StateT Memory IO ()`. Originally, the function operated on the `IO Monad`. It was able to take in text to parse, and output responses. I decided to use `States` to give LGPT a memory. However, I quickly ran into a problem: I could not use both `Monads` at the same time.

I did some research, and found `StateT`, a monad transformer. It takes a base monad (`IO`) and adds state-handling capabilities on top of it, fusing them into a single monadic context. I defined my `Memory` type alias - a map containing pairs of strings, which allowed me to associate a key to a value, perfect for remembering things. `()` is simply the return type, which the function has none. Thus, the type becomes `StateT Memory IO ()`, which is able to perform both `IO` and `Stateful` actions! It seemed perfect as it was a (mostly) simple way to be able to use `States` while continuing to use `IO`!

As a result of having the `IO` actions wrapped, I couldn't execute any further `IO` functions normally. Instead, I had to use `liftIO`, which lifts the action into a stateful type, so it can be performed. This is why `liftIO` is seen to wrap every single `IO` function I run.

As an extra feature I decided to store everything in a hidden text file in the `/src` directory called `.memory.txt`. Every time the program saves something to memory, I overwrite the text file with the current state. Every time the program is run, it checks the text file and passes the contents into the `runREPL` function to have an initial state. This allows for a proper memory.

3 Implementation decisions

3.1 Today

To get the current time, I used the handy `getCurrentTime` function from the `Data.Time.Clock` package.

However, it doesn't return it in the correct form, so I then have to further apply `fmap` to it, along with the `utcDay` function. This returns the current day as a `Day` data type, which is perfect for the responses I need to provide.

3.2 Calculations

My parser and response functions have a lot going on. First, I defined a `Term` data type which either contains a `That`, or a `Value Int`. This allows me to easily incorporate the result of the previous calculation into the current one. The parser (`parseExpr`) just goes through the input, and transforms it into a `Request` type in the form `Expression Term [(String, Term)]`, where the first `Term` is the first number in the calculation, and the rest is a pair of an operator and a number.

Then, I run `resolveTerm`, and then I fold everything. There is a separate resolver function which takes a `Term` and transforms it into a `Maybe Int`. Then, during the fold, I extract the integer, or if there's an issue, I respond with an error message.

3.3 Forgetting

I thought that if the bot can remember things, we should be able to ask it to forget things! Therefore, I added two Request types: Forget `String` and ForgetAll. When using Forget, we use the attached string to look it up in the map, and remove it completely. For ForgetAll, we just use `put Map.empty` to wipe the state and then update the text file normally. To use: ask "Forget about <thing>." or "Forget about everything."

3.4 Reading Files

Using `System.FilePath` and `System.Directory` I was able to parse a request to read a specific file, using a path we provide, and output the contents. I ensured that the user can use the tilde prefix by using:

```
1 home <- liftIO getHomeDirectory
2 let normalisedPath = intercalate home (splitOn "~" path)
3 if (home `isPrefixOf` normalisedPath) then ...
```

Of course, I then included the necessary logic to process errors, including if the file format cannot be decoded. To use, ask "Output the contents of <path>."

3.5 Remembering: QOL updates

When I asked the bot to remember something about "my <item>" it would think it owns the item. Well, the program cannot own anything because it is a computer program, which (as of April 2026) has no rights.

To fix this, I slightly modified the parser for remembering things by replacing the word "my" with "your" when storing the information. Now, we can have ownership again:

```
1 > Remember that my coursework is perfect.
2 Okay.
3 > Tell me about my coursework.
4 Sure - your coursework is perfect.
```

I also added a feature so that the program could remember multiple things about a single item. I do this simply by concatenating the previous things with ", and <new thing>".

```
1 > Remember that my coursework is written in Haskell.
2 Okay.
3 > Tell me about my coursework.
4 Sure - your coursework is perfect, and written in Haskell.
```

Also, since the result of the last calculation is stored under a key `_last_result_` in memory, I made sure it cannot be overwritten or recalled by the user.

3.6 Dictionary

I used the **Dictionary API**[1] to create a Define request. It simply takes the word to be defined into the input, concatenates it to the URL, forms the API request with `Network.HTTP.Simple's` `parseRequest` (not to be confused with our own) and sends it off. The response is in JSON, so I created multiple data types that derive the generic type class from `GHC.Generics`, and I ensure they become instances of `FromJSON`. This allows us to view the response as a multi dimensional list of meanings, which is returned by running `getResponseBody`. We then pattern match several levels into the list until we find the first meaning, and return it. Of course, if the word was not found or there were API errors, we show an appropriate error message. Most of this is done in a separate function. To use: ask "Define <word>."

3.7 Easter egg!

Because I was procrastinating on writing the report, I used `randomRIO` from the `System.Random` package to have a specific prompt that 90% of the time is given a normal response. 10% of the time, the response is... interesting. Try asking: "What are you?" (It's not malicious, just text.)

References

[1] MeetDeveloper. Free dictionary api. <https://dictionaryapi.dev/>. Accessed: April 2026.